

INTRODUCERE ÎN SISTEME CU MICROPROCESOARE

Partea a doua

1

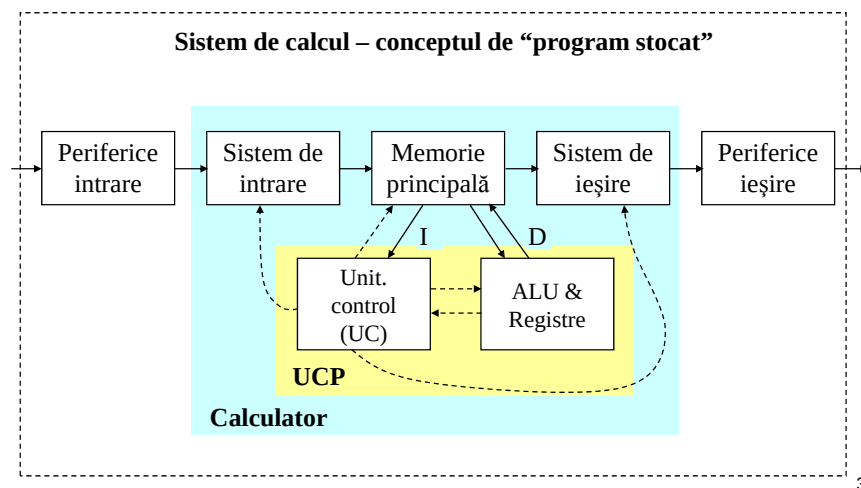
Reprezentare structurală a calculatorului Arhitectura von Neumann (definită în 1945)

Principii:

- **Datele și instrucțiunile** sunt stocate în aceeași **memorie principală** (**conceptul de program stocat**);
- Conținutul memoriei este **adresabil la nivel de locație** (fără a se considera ce informație este stocată: date sau cod);
- **Instrucțiunile** sunt **executate secvențial** (una câte una în ordinea locațiilor de memorie în care sunt stocate) cu excepția faptului când acest lucru este precizat explicit.
- Calculatorul conține următoarele subsisteme: unitate de control, unitate aritmetică și logică (ALU), sistem IO, memorie
- Calculatoarele cu arhitectură von Neumann sunt calculatoare de uz general. Ele pot rezolva diverse probleme în funcție de programul executat.

2

Arhitectură de uz general secvențială (von Neumann) – reprezentare structurală



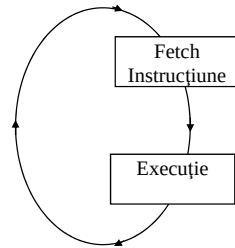
Reprezentare structurală von Neumann

- Funcția principală a UCP este să execute instrucțiunile aduse (“**fetch**”) din memoria principală.
- O instrucțiune codifică pentru UCP o operație de bază (aritmetică, logică, transfer de date la / de la memorie, etc.) ce trebuie efectuată.
- UC interpretează (decodifică) instrucțiunea curent adusă și comandă celelalte unități funcționale pentru a se putea executa instrucțiunea.
- UCP conține un set de registre folosite ca locații de stocare temporară pentru rezultate intermediare și date imtens utilizate de program.
- Fiecare instrucțiune este executată într-o secvență de pași, numită **ciclu instrucțiune**.

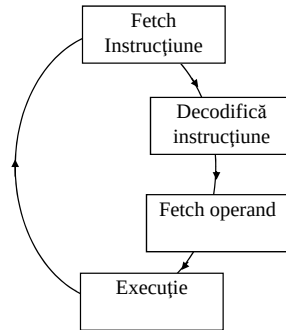
4

Reprezentare a ciclului instrucțiune

- Reprezentare simplificată a ciclului instrucțiune:



- O reprezentare extinsă a ciclului instrucțiune :

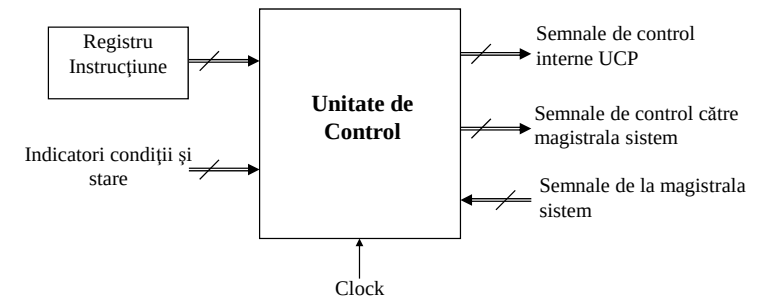


Pentru rularea programului UCP are o funcționare ciclică la fiecare instrucțiune executată

5

Unitate de Control

- Unitatea de control (UC) este “creierul” procesorului
- UC controlează și sincronizează toate elementele din interiorul UCP și interfețele către calea de date externă.



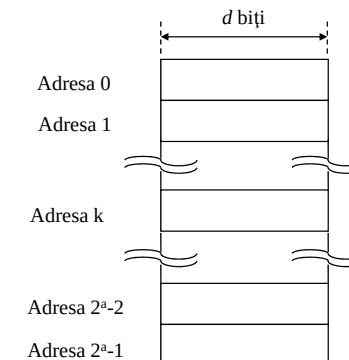
6

Unitate de prelucrare (ALU și Registre)

- Unitatea aritmetică și logică (ALU), realizează operații **logice** sau **aritmetice** cu operanzii adresați de UC
- Înainte de prelucrare, operanzii se stochează într-un set de **registre de uz general**, folosite ca memorie temporară
- Registrele reprezintă o memorie locală UCP, de foarte **mare viteză**
 - Setul de registre de uz general poate fi folosit însă și pentru salvarea diferitelor informații privind adresarea memoriei principale.
- În funcție de rezultatul operațiilor efectuate, ALU setează anumiți **indicatori de condiții** (indicatori de stare, fanioane)

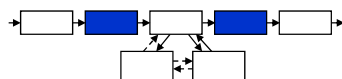
7

MEMORIA PRINCIPALĂ (INTERNĂ)



8

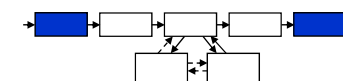
Sistem de Intrare / Ieșire



- Sistemul de IO conține locații adresabile (**port-uri**) prin care calculatorul face schimb de informație cu lumea externă. Sistemul conține circuite pentru funcțiile:
 - schimbare **format** date (serial / paralel)
 - **stocare temporară** de date (latch)
 - **verificare a corectitudinii** datelor transferate
 - **sincronizare** între periferic și UCP (datorită diferențelor mari de viteză de funcționare).
 - logică de **decodificare**

9

Periferice



- **Dispozitive (periferice) de intrare** – recepționează informația din mediul extern (prin diverși senzori) și o transformă ca **natură fizică** și **format de reprezentare** ↔ compatibilitate cu nivelurile de tensiune recunoscute de calculator (ca valori binare).
- **Dispozitive (periferice) de ieșire** – operație inversă dispozitivelor de intrare. Prelucreează datele din calculator și le transformă în informație acceptabilă la dispozitivul de ieșire.

10

REPREZENTAREA FUNCȚIONALĂ A UNUI CALCULATOR

- Un calculator poate fi descris și prin funcțiile - operațiile pe care le pune la dispoziția utilizatorului
- Circuitele electronice ale unui calculator recunosc și execută doar un set limitat de *instrucțiuni elementare*, codificate în formă binară
- Instrucțiunile recunoscute sunt rareori mai complicate decât:
 - adună două numere
 - verifică dacă un număr este egal cu zero
 - copiază date dintr-o zonă a memoriei calculatorului în altă zonă.

11

LIMBAJ DE PROGRAMARE

- Instrucțiunile elementare, recunoscute de un procesor formează **limbajul mașină** al procesorului
- Notatie: limbaj **L0** ce corespunde mașinii fizice **M0**.
- Programatorul scrie programele sale într-un limbaj **L1**, format din instrucțiuni simbolice (succesiuni de litere și cifre) și mai apropiat de limbajul uman
- Pentru a putea executa acest program pe mașina M0 fiecare instrucțiune din L1 a programului trebuie **translatată** în instrucțiuni din limbajul L0
- De exemplu programatorul poate scrie în L1 instrucțiunea:
add al,6
- iar programul de traducere va genera succesiunea binară, corespunzătoare lui L0:
0000 0100 0000 0110

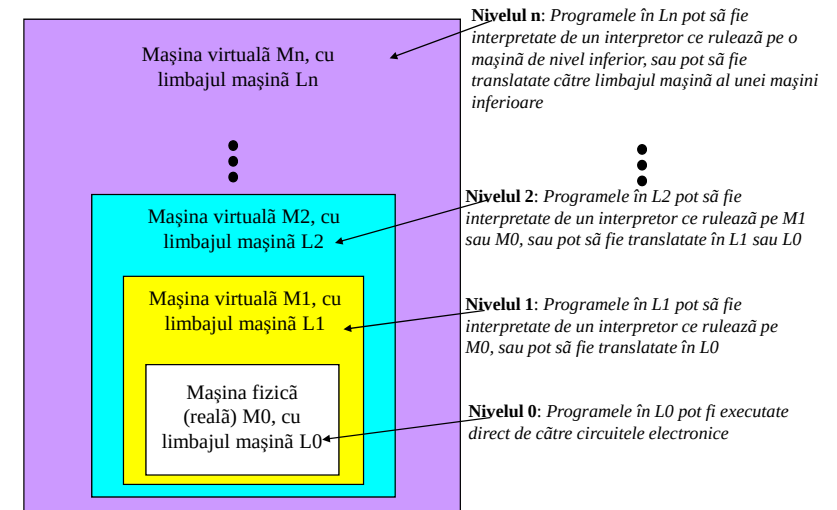
12

MAȘINI VIRTUALE

- Funcțional, se poate spune că limbajul L1 corespunde unei mașini virtuale programabile numită M1. Numele acestui limbaj simbolic de programare este "limbaj de asamblare", iar programul de traducere este numit "asamblor".
- Traducerea: - tot programul scris în L1 este transformat în program scris în L0 (executabil), iar programul în L1 este abandonat.
- programul în L0 este încărcat în memoria calculatorului și executat
- Exemple de programe **translator**: **compilator**, **asamblor**.
- **Interpretorul** este un program în L0 care rulează pe M0 și care preia instrucțiunile programului în L1 ca pe date de intrare. Interpretorul citește și decodifică fiecare instrucțiune din L1 și apoi trece imediat la execuția acesteia. La interpretare nu se generează un program executabil ca la traducere
- Limbaje de programare mai apropiate de limbajul uman: L2, L3, ... (pentru mașini virtuale M2, M3, ...)

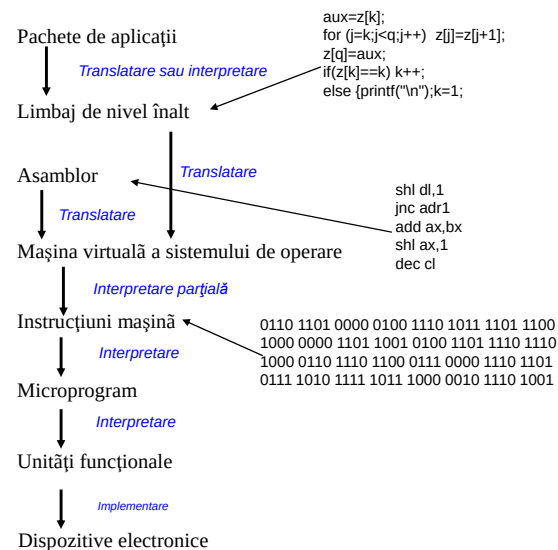
13

Calculator = succesiune de mașini virtuale



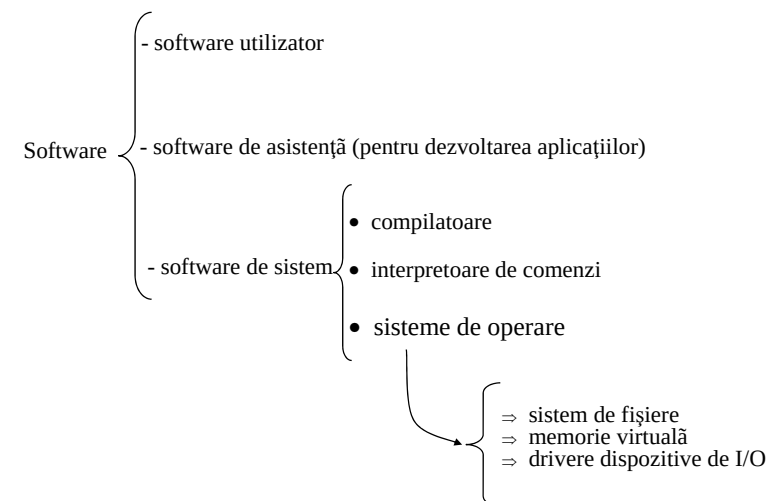
14

Ierarhie de mașini virtuale la GPC micro-programat



15

SOFTWARE



16

Evaluarea performanțelor

17

TIMPUL - MĂSURĂ A PERFORMANȚELOR CALCULATORULUI

- Performanțele procesorului unui calculator numeric se evaluează în primul rând prin determinarea timpului de execuție al programelor
- *Timpul* este măsura ce indică performanța
- Comparând două mașini de calcul, concluzia de genul "Mașina A este mai rapidă decât mașina B" se referă la faptul că timpul de execuție al aceluiași program pe mașina A este mai mic decât pe mașina B
- Cele spuse mai sus sunt adevărate pentru un calculator care execută un singur program de testare, fără întreruperi de la lansare, până la terminarea sarcinii de lucru

18

TIMPUL - MĂSURĂ A PERFORMANȚELOR CALCULATORULUI

- Pentru un sistem cu multi-programare, UCP poate abandona temporar un program (întreruperea execuției) în timp ce se așteaptă operații de I/O și să se treacă la execuția altui program
 - Ca urmare, în măsurarea timpului de execuție al unui program pe un anumit procesor trebuie ținut cont și de eventuala întrerupere temporară a programului
- Din punctul de vedere al utilizatorului timpul este un *timp de răspuns* (de la lansare până la terminarea execuției) sau un *timp de așteptare*, care include și intervalele de timp când se rulează altceva, prin partajarea timpului procesorului
- Se introduce noțiunea de *timp al UCP* (T_{UCP}), care este o măsură a timpului efectiv cât procesorul (UCP) rulează programul de testare a performanțelor
 - Acest timp reprezintă timpul cât UCP realizează prelucrări fără a include așteptarea operațiilor de I/O și rularea altor programe
 - Timpul T_{UCP} include însă și intervalele de timp ce corespund timpului UCP consumat de către sistemul de operare pentru realizarea unor sarcini cerute de programul de test.

TIMPUL - MĂSURĂ A PERFORMANȚELOR CALCULATORULUI

- Utilizatorul calculatorului este interesat în primul rând de reducerea *timpului de execuție* (sau a *timpului de răspuns* între momentele startului și terminării unei sarcini de lucru), sau altfel spus în micșorarea timpului de așteptare (exprimat adesea ca latență)
- Managerul centrului de calcul este interesat în creșterea *performanțelor globale* ale sistemului de calcul exprimate prin denumirea "*throughput*" (putere de calcul).
 - De obicei termenii "timp de răspuns" și "throughput" se utilizează când se discută o întreagă sarcină de calcul.
 - Atunci când se analizează performanțele sistemului de memorie se folosesc termenii "*bandwidth*" (*lărgime de bandă*) și "*latency*" (*timp de așteptare, întârziere în răspuns*)

20

Timp de răspuns (timp de execuție, latență):

- După cât timp calculatorul răspunde la datele introduse de utilizator, sau după cât timp termină programul
 - Un utilizator vede rezultatul după 5 secunde de la introducerea numelui unei biblioteci / baze de date
 - Un program de simulare se termină într-o oră
 - Cantitatea de timp pentru a produce o linie de cod
 - Cantitatea de timp pentru transmiterea unui bit printr-un fir

21

Throughput:

- Cantitatea de muncă efectuată într-un timp dat (throughput, pentru că sistemul rulează mai multe sarcini simultan)
 - Un web server răspunde la maxim 5 milioane de cereri pe secundă
 - Numărul de linii de cod executate pe oră
 - Numărul de biți pe secundă transferați printr-un fir
 - Alte măsuri: **MIPS**, **MFLOPS**, **frecvență de clock**, interesante atunci când performanțele sunt direct legate de timpul de răspuns, sau throughput

22

Mașina A este *de n ori* mai rapidă decât mașina B

$$\frac{Timp_executie_B}{Timp_executie_A} = n$$

$$\frac{Timp_executie_B}{Timp_executie_A} = \frac{1/Performanta_B}{1/Performanta_A} = \frac{Performanta_A}{Performanta_B} = n$$

23

Comparație procentuală

- Mașina A este cu p% mai rapidă decât mașina B

$$\frac{Timp_executie_B}{Timp_executie_A} = 1 + \frac{p}{100}$$

$$\frac{Performanta_A}{Performanta_B} = 1 + \frac{p}{100}$$

$$p = 100 \cdot \left(\frac{Performanta_A - Performanta_B}{Performanta_B} \right)$$

24

Exemplul 1:

- Dacă mașina A rulează un program de test în 10 secunde, iar mașina B rulează același program în 15 secunde, care din următoarele răspunsuri este corect?

? A este cu 50% mai rapidă ca B

? A este cu 33% mai rapidă ca B

25

Exemplul 1:

Răspuns:

- Mașina A este cu p% mai rapidă ca B, exprimat ca:

$$\frac{Timp_executie_B}{Timp_executie_A} = 1 + \frac{p}{100}$$

sau

$$p = 100 \cdot \left(\frac{Timp_executie_B - Timp_executie_A}{Timp_executie_A} \right) = 100 \cdot \left(\frac{15 - 10}{10} \right) = 50$$

A este cu 50% mai rapidă decât B.

26

Calcul T_{UCP}

$$T_{UCP} = NrCLK_p \times T_{clk} = NrI \times CPI \times T_{clk}$$

- $NrCLK_p$ = număr impulsuri de ceas pe program
- NrI = numărul de instrucțiuni din program
- CPI = numărul mediu de perioade de ceas pe instrucțiune; poate fi calculat prin împărțirea numărului de perioade de ceas pentru un program, la numărul de instrucțiuni din program.
- T_{clk} = perioada impulsului de ceas a procesorului ($1/f_{clk}$)

27

Exemplu

- Dacă frecvența ceasului unui procesor este de 1 MHz, iar rularea unui program necesită 5 milioane de perioade de tact pentru execuție completă, care este timpul de execuție al programului?

$$\begin{aligned} T_{UCP} &= NrI \times CPI \times T_{clk} \\ &= Perioade_tact_pe_program \times T_{clk} \\ &= \frac{Perioade_tact_pe_program}{f_{clk}} \end{aligned}$$

$$T_{UCP} = \frac{5 \times 10^6}{1 \times 10^6} = 5 \text{ sec unde}$$

28

Comentarii T_{UCP}

$$T_{UCP} = NrI \times CPI \times T_{clk}$$

- Încercând să comparăm diferite sisteme, apar următoarele problemele:
 - frecvențele de ceas sunt diferite pentru diferite calculatoare
 - arhitecturile celor două calculatoare comparate pot fi diferite, conducând la valori diferite pentru componentele din relația timpului de execuție pe UCP.
- Pentru a putea face o comparație cât de cât acceptabilă, trebuie rulat același program de testare a performanțelor pentru ambele calculatoare care se compară.

29

Micșorarea componentelor T_{UCP}

$$T_{UCP} = NrI \times CPI \times T_{clk}$$

- Numărul mediu de perioade de ceas pe instrucțiune (CPI) este determinat de:
 - arhitectura procesorului, prin componentele: set de instrucțiuni și modul de organizare
 - viteza de lucru a procesorului cu memoria principală
- Numărul de instrucțiuni (NrI), depinde de
 - tipul programului rulat
 - arhitectura setului de instrucțiuni a procesorului (în sens contrar față de CPI)
 - un număr mai mic de instrucțiuni $\Rightarrow$ instrucțiuni mai complexe (CPI mare)
 - structura și calitatea compilatorului folosit la translatarea din program sursă în program obiect
- Frecvența impulsului de ceas a UCP depinde de
 - tehnologia de realizare a procesorului
 - organizarea internă a acestuia

30

Rezumat influențe termeni

$$T_{UCP} = NrI \times CPI \times T_{clk}$$

	Inst Count (NrI)	CPI	Clock Rate (f_{CLK})
Program	X		
Compiler	X	(X)	
Inst. Set.	X	X	
Organization		X	X
Technology			X

31

Exemplu

- Într-un program de testare a performanțelor (benchmark) cu 250 instrucțiuni, există următoarele tipuri de instrucțiuni:
 - 100 instrucțiuni de tip load/store - 5 perioade de ceas/instr
 - 80 instrucțiuni de transfer între registre interne ale UCP - 1 perioadă de ceas.
 - 50 instrucțiuni de adunare sau scădere - 3 perioade de ceas
 - 20 instrucțiuni de înmulțire cu întregi - 50 perioade de ceas
- Calcul CPI pentru programul de test.

$$CPI = \frac{(100 \times 5) + (80 \times 1) + (50 \times 3) + (20 \times 50)}{250} = \frac{1730}{250} = 6.92$$

32

Calcul număr impulsuri de ceas/program

$$NrCLK_p = \sum_{i=1}^n (CPI_i * I_i)$$

- unde:
 - I_i indică de câte ori este executată instrucțiunea de tip i într-un program
 - CPI_i - numărul mediu de perioade de ceas necesare pentru execuția instrucțiunii i

• $\Rightarrow$

$$T_{UCP} = T_{clk} * \sum_{i=1}^n CPI_i * I_i$$

33

CPI funcție de “frecvența instrucțiunilor”

$$CPI = \frac{NrCLK_p}{NrI} = \frac{\sum_{i=1}^n (CPI_i * I_i)}{NrI}$$

$$CPI = \sum_{i=1}^n \left(CPI_i * \frac{I_i}{NrI} \right)$$

$$CPI = \sum_{i=1}^n CPI_i * F_i$$

- De investit (optimizat) acolo unde se pierde timpul, dacă frecvența de apariție a instrucțiunilor respective este semnificativă în programe
- De investit în optimizarea instrucțiunilor cu frecvență mare și nu în cele folosite rar

34

MIPS - Milioane de Instrucțiuni Pe Secundă

$$MIPS = \frac{f_{clk}}{CPI \times 10^6}$$

- Avantajul introducerii parametrului MIPS constă în exprimarea ușoară și în faptul că este deosebit de sugestiv pentru clienții cumpărători de mașini de calcul
- MIPS prezintă însă dezavantaje atunci când este folosită pentru a compara diferite mașini, pentru că:
 - Valoarea MIPS este dependentă de setul de instrucțiuni, făcând dificilă și neconcludentă compararea între calculatoare cu seturi diferite de instrucțiuni;
 - Valoarea MIPS, pentru același calculator, variază în funcție de programele rulate. La programele cu operații complexe valoarea medie CPI este mare;
 - Valoarea MIPS poate varia uneori invers proporțional cu performanța
 - Exemplul clasic folosit pentru a demonstra aceasta, se referă la operațiile cu numere reale, reprezentate în virgulă mobilă

35

Exemplu de interpretare greșită prin MIPS

- Se compară două mașini, notate cu A și B
- Mașina A recunoaște o instrucțiune specială de înmulțire a întregilor cu semn, care consumă 50 de perioade de ceas
- Mașina B nu are instrucțiune specială de înmulțire, dar o realizează prin instrucțiuni simple de adunare, deplasare, comparare, fiecare dintre instrucțiunile simple consumând două perioade de ceas
- Pentru operația de înmulțire, la o frecvență de ceas pentru ambele mașini de 200 MHz, rezultă valorile MIPS astfel:
 - Mașina A = $200/50$ MIPS = 4 MIPS
 - Mașina B = $200/2$ = 100 MIPS
- ceea ce ar indica că mașina B este de 25 ori mai rapidă, deși nu aceasta este realitatea.

36

MFLOPS - milioane de operații în virgulă mobilă pe secundă

$$MFLOPS = \frac{\text{Nr. operatii în VM din program}}{T_{UCP} \times 10^6}$$

- Bazată pe operații și nu pe instrucțiuni

37

MFLOPS - milioane de operații în virgulă mobilă pe secundă

- MFLOPS nu este pe deplin edificatoare în compararea performanțelor
- Operațiile în virgulă mobilă implementate pe diferite mașini nu sunt aceleași
- Unele procesoare pot executa, prin implementare cablată, operații în virgulă mobilă de împărțire, radical, sinus etc., iar altele pot recunoaște doar instrucțiuni aritmetice simple ca adunare și scădere cu numere reale
- Mai mult, operațiile în virgulă mobilă nu au aceeași complexitate
 - unele se efectuează mai rapid, altele mai lent
 - De exemplu, pe aceeași mașină, vom obține o valoare mai mare pentru MFLOPS, dacă programul de test are doar adunări în virgulă mobilă, față de cazul când programul de test are operații complexe de împărțire, sinus, radical

38

BENCHMARKS - Programe etalon de testare a performanțelor

- Programele de test se pot focaliza pe aspecte specifice ale sistemului
 - Unitățile FP
 - ALU (întregi)
 - Sistem de memorie
 - I/O
 - Sistem de operare

39

Benchmarks - categorii

- **Programe reale**
 - compilatoare de C, programe de procesare a textelor, programe de proiectare asistată
 - Testarea se face în condiții similare privind intrările, ieșirile și setarea opțiunilor
- **Aplicații modificate**
 - Testare prin blocuri componente ale unor aplicații
 - Programe de test controlate prin fișiere de comenzi
- **Nuclee (kernels)** din programe reale
- **Toy benchmarks**
 - sunt programe scurte (maxim 100 de linii de cod) care produc un rezultat cunoscut înainte de rulare, folosite doar pentru a testa viteza de execuție
- **Programe de test sintetice (synthetic benchmarks)**
 - programe create artificial, pentru testare
 - Exemple: Dhrystone și Whetstone

40

SPEC

- SPEC - The Standard Performance Evaluation Corporation, ca organizație non-profit formată cu scopul de a *stabili, menține și garanta* seturi standardizate de programe de test (benchmarks) ce pot fi aplicate la noile generații de calculatoare de înaltă performanță
 - Organizația s-a numit inițial "System Performance Evaluation Cooperative" și ea cuprindea firmele HP, DEC, MIPS și Sun
- SPEC - standardizarea de programe de test în multe domenii: sisteme de calcul, rețele de calculatoare, sisteme embedded, web-servers etc.
- SPEC nu realizează testare, ea doar dezvoltă seturi de programe de test și de asemenea analizează și publică rezultatele propuse/obținute de membrii organizației și alte instituții licențiate pentru aceasta

41

SPEC

- Consorțiul stabilește *setul de programe de test* (doar programe reale, nuclee, sau aplicații modificate) și intrările folosite la rularea acestor programe
- Principala caracteristică a programelor de test SPEC este că sunt formate din *seturi (suite)* de aplicații pentru testarea procesoarelor
 - Avantajul setului de programe de test este dat de faptul că eventuala slăbiciune a uneia dintre aplicațiile de testare poate fi compensată rulând celelalte aplicații din setul etalon de testare (benchmark).

42

SPEC

- În cazul programelor pentru testarea UCP, există în prezent mai multe generații de seturi de programe de test, pentru operare în virgulă fixă (INT - întregi) și virgulă mobilă (reale, FP - Floating Point): SPEC89, SPEC92, SPEC95, SPEC2000 SPEC2009
- De exemplu SPEC2009 oferă o suită de programe de test, dintre care menționăm:
 - CPU
 - Graphics/Workstations
 - MPI/OMP (suită de programe pentru evaluarea performanței sistemelor paralele ce utilizează aplicații de tip MPI (Message Passing Interface), sau programe de interfață cu aplicația OpenMP pentru platforme de calcul paralel cu memorie partajată
 - Java Client/Server
 - Mail Servers
 - Network File System
 - Power
 - SIP (servere ce folosesc Session Initiation Protocol - SIP)
 - SOA (măsurare de performanțe pentru baze de date și hardware distribuit, al aplicațiilor bazate pe Service Oriented Architecture - SOA)
 - Web Servers
- <http://www.spec.org>

43

Legea lui Amdahl

- Permite calcularea câștigului în performanță obținut după îmbunătățirea unei componente a unui calculator
- Legea stabilește valoarea creșterii globale a vitezei sistemului în condițiile utilizării unei creșteri a vitezei doar pentru una din componentele sistemului
- Legea permite calculul valorii numerice a creșterii în viteză a sistemului pe baza îmbunătățirii unei caracteristici particulare ("speedup", ca raport între timpul consumat înainte de îmbunătățire și timpul consumat după îmbunătățire)

44

Legea lui Amdahl

- Dacă se crește viteza de execuție de n ori doar pentru o fracțiune F din timpul de execuție al unui program nu înseamnă că am crescut viteza de execuție a programului de n ori.
- Dacă se știe frecvența cu care se utilizează o anumită îmbunătățire locală dintr-un sistem, se poate calcula cu cât crește viteza întregului sistem datorită îmbunătățirii
- Îmbunătățirea locală pentru o unitate centrală de procesare se referă de exemplu la: dublarea frecvenței de ceas, reducerea la jumătate a timpului mediu de acces la memoria principală, dublarea vitezei de lucru la circuitul înmulțitor etc.
- Dacă se dorește o creștere globală cât mai mare a performanțelor unui sistem trebuie în primul rând crescute performanțele subsistemelor utilizate în majoritatea timpului de execuție

45

Legea lui Amdahl

- Pentru a exprima analitic legea lui Amdahl vom nota cu T_{nou} timpul de execuție consumat după îmbunătățirea unei componente și cu T_{vechi} timpul de execuție înainte de îmbunătățire, raportul lor reprezentând creșterea în viteză a sistemului.

$$Speedup = \frac{T_{vechi}}{T_{nou}} = \frac{Performanta_{noua}}{Performanta_{veche}} \Rightarrow Speedup = \frac{T_{vechi}}{T_{nou}} = \frac{n}{n - nF + F}$$

$$\Rightarrow Speedup = \frac{1}{(1 - F) + \frac{F}{n}}$$

T_{nou} = timpul de execuție consumat după îmbunătățirea de n ori a unei componente

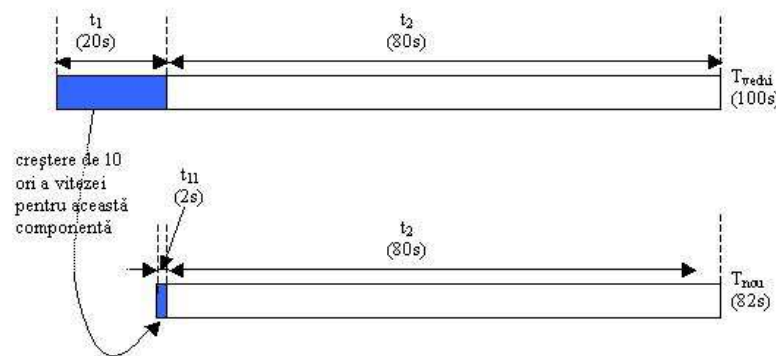
T_{vechi} = timpul de execuție înainte de îmbunătățire

F = fracțiunea de timp cât este utilizată componenta îmbunătățită

n = de câte ori au fost crescute performanțele unei componente.

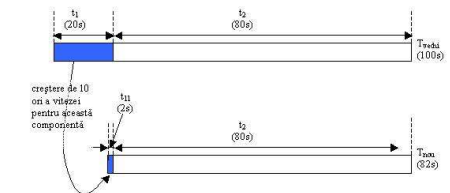
46

Legea lui Amdahl – justificare ecuație



47

Legea lui Amdahl – justificare ecuație



$$T_{vechi} = t_1 + t_2$$

$$t_1 = F \times T_{vechi}$$

$$t_2 = (1 - F) \times T_{vechi}$$

$$F = 20\%$$

48

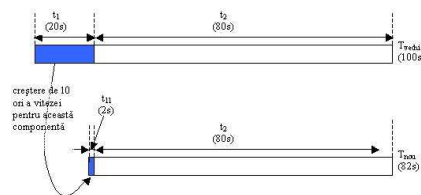
Legea lui Amdahl – justificare ecuație

$$T_{nou} = t_{11} + t_2$$

$$= \frac{t_1}{n} + t_2$$

$$T_{nou} = \frac{F \times T_{vechi}}{n} + (1 - F) \times T_{vechi}$$

$$\frac{T_{nou}}{T_{vechi}} = \frac{F}{n} + (1 - F)$$



49

Exemplul 1

- Un program de test rulează 100 de secunde pe un calculator uniprocessor
- 10% din program poate fi paralelizat pe un multiprocesor (F=0.1)
- Presupunem un multiprocesor cu 10 procesoare (n=10)
- Creșterea în viteză generală:

$$Speedup = \frac{1}{(1 - 0.1) + \frac{0.1}{10}} = \frac{1}{0.01 + 0.9} = \frac{1}{0.91} = 1.1$$

50

Exemplul 2

- La un procesor unitatea aritmetică în virgulă mobilă este îmbunătățită, astfel că rulează de 8 ori mai repede. Calculați creșterea în viteză a procesorului, pentru un program la care doar 20% din instrucțiuni folosesc operanzi în virgulă mobilă
- F = 20%
- n = 8

$$Speedup = \frac{1}{(1 - 0.2) + \frac{0.2}{8}} = 1.212$$

51